Laporan Tugas Kecil 1 IF2211

Strategi Algoritma Semester II tahun 2025/2026

Penyelesaian Permainan Queens Linkedin

v1.0

DEADLINE: RABU, 2026/02/18 11:59 WIB

Biodata Penulis

Nama : Aziza Dharma Putri

NIM : 13524017

Laboratorium Ilmu dan Rekayasa Komputasi

Program Studi Teknik Informatika

Sekolah Teknik Elektro dan Informatika

Institut Teknologi Bandung

DAFTAR ISI

PENDAHULUAN

Konteks Persoalan

Program ini dibuat untuk menyelesaikan permainan Queens LinkedIn. Diberikan papan berukuran N × N yang sudah dibagi menjadi N region berwarna. Tugasnya adalah menempatkan tepat satu queen di setiap baris, setiap kolom, dan setiap region. Selain itu, ada aturan tambahan yaitu tidak boleh ada dua queen yang saling bertetangga dalam delapan arah (atas, bawah, kiri, kanan, dan keempat diagonal).

Program memiliki dua mode penyelesaian, yaitu Pure Brute Force dan Optimized Backtracking.

Mode 1: Pure Brute Force (**dfsPure**)

Mode ini menggunakan pendekatan brute force murni. Artinya, program mencoba semua kemungkinan penempatan queen tanpa melakukan pengecekan di tengah proses. Validasi baru dilakukan setelah seluruh papan terisi.

Algoritma berjalan baris demi baris secara rekursif, mulai dari baris 0 hingga baris N−1. Pada setiap baris, semua kemungkinan kolom dari 0 sampai N−1 dicoba satu per satu. Untuk setiap kolom yang dicoba, semua sel pada baris tersebut dikosongkan terlebih dahulu, lalu queen ditempatkan di posisi (row, c). Setelah itu penghitung jumlah kasus dinaikkan dan fungsi rekursif dipanggil untuk mengisi baris berikutnya.

Jika nilai row sudah sama dengan N, berarti semua baris telah terisi satu queen. Pada titik ini fungsi `isCompleteValid()` dipanggil untuk memeriksa apakah konfigurasi yang terbentuk valid. Pemeriksaan meliputi tiga hal: tidak ada dua queen di kolom yang sama, tidak ada dua queen di region yang sama, dan tidak ada dua queen yang bertetangga dalam delapan arah.

Jika konfigurasi valid, solusi ditemukan dan proses berhenti. Jika tidak valid, dilakukan backtrack dengan mencabut queen terakhir lalu mencoba kolom berikutnya.

Karena tidak ada pengecekan selama proses penempatan berlangsung, seluruh cabang rekursi akan dibangun hingga kedalaman penuh N. Jumlah konfigurasi yang diperiksa adalah N^N. Sebagai contoh, untuk N = 8 terdapat lebih dari 16 juta kemungkinan konfigurasi. Kompleksitas waktu pendekatan ini adalah O(N^N), sehingga menjadi sangat besar untuk nilai N yang lebih tinggi.

Mode 2: Optimized Backtracking (**dfsOptimized**)
Mode ini masih menggunakan pendekatan rekursif baris demi baris, tetapi ditambahkan mekanisme pruning atau pemangkasan cabang yang sudah pasti tidak valid.

Perbedaannya terletak pada saat sebelum queen ditempatkan. Sebelum menaruh queen di posisi (row, c), program memanggil fungsi `canPlace(row, c)` untuk mengecek apakah posisi tersebut memenuhi semua kendala.

Ada tiga kendala yang diperiksa. Pertama adalah kendala kolom, yaitu memastikan kolom c belum ditempati queen pada baris sebelumnya. Kedua adalah kendala ketetanggaan, yaitu memeriksa delapan sel di sekitar posisi tersebut untuk memastikan tidak ada queen yang bersebelahan. Ketiga adalah kendala region, yaitu memastikan region tempat sel tersebut berada belum memiliki queen.

Jika salah satu kendala tidak terpenuhi, posisi tersebut langsung dilewati tanpa melanjutkan rekursi. Jika ketiganya terpenuhi, barulah queen ditempatkan dan algoritma melanjutkan ke baris berikutnya.

Ketika row sudah mencapai N, semua baris telah terisi dan konfigurasi otomatis valid karena setiap langkah penempatan sudah diverifikasi sejak awal. Tidak diperlukan lagi pengecekan tambahan seperti pada mode brute force.

Dengan adanya pruning, banyak cabang rekursi dihentikan lebih awal sebelum mencapai kedalaman N. Secara teoritis kompleksitasnya mendekati O(N!), dan dalam praktiknya sering kali lebih kecil lagi karena adanya tambahan kendala region.

IMPLEMENTASI

Alur Program

1. Pengguna memuat puzzle melalui GUI (file .txt atau gambar .png/.jpg).
2. ImageIO mem-parsing input menjadi objek Puzzle yang berisi: ukuran N, matriks regionId[N][N], dan array regionColors.
3. Puzzle divalidasi oleh Puzzle::isValid() untuk memastikan N region unik dan semua ID valid.
4. Pengguna memilih mode (Pure Brute Force atau Optimized) lalu klik Solve.
5. Solver berjalan di thread terpisah (QThread) agar GUI tetap responsif.
6. Setiap 2000 kasus dicoba, Solver mengemit sinyal progress() untuk memperbarui tampilan papan secara real-time.
7. Setelah selesai, Solver mengemit sinyal finished() dengan hasil Puzzle, SolveStats (waktu dan jumlah kasus), dan pesan status.
8. Pengguna dapat mengekspor solusi sebagai file .txt atau gambar .png

Struktur File

```
├── CMakeLists.txt
├── Makefile
├── README.md
├── .gitignore
├── src/
│   ├── main.cpp
│   ├── MainWindow.h
│   ├── MainWindow.cpp
│   ├── BoardWidget.h
│   ├── BoardWidget.cpp
│   ├── Puzzle.h
│   ├── Puzzle.cpp
│   ├── Solver.h
│   ├── Solver.cpp
│   ├── ImageIO.h
```

```
│   ├── ImageIO.cpp
│   └── txt2img.cpp
├── test/
├── doc/
├── build/
└── bin/
```

Source Program

1. main.cpp

   File ini adalah titik masuk (entry point) program. Tugasnya sederhana, yaitu menginisialisasi QApplication, mendaftarkan tipe Puzzle dan SolveStats ke sistem metatype Qt menggunakan qRegisterMetaType agar kedua tipe tersebut dapat dikirimkan melalui sinyal-slot lintas thread, kemudian membuat dan menampilkan jendela utama MainWindow.

```
#include "MainWindow.h"
#include "Puzzle.h"
#include "Solver.h"
#include <QApplication>
int main(int argc, char *argv[]) {
   QApplication app(argc, argv);
   qRegisterMetaType<Puzzle>();
   qRegisterMetaType<SolveStats>();
   MainWindow w;
   w.show();
   return app.exec();
}
```

2. MainWindow.h dan MainWindow.cpp

   File ini mengimplementasikan jendela utama aplikasi dan bertindak sebagai koordinator seluruh komponen. Di sini dibangun layout GUI lengkap: panel kiri berisi BoardWidget untuk visualisasi papan, dan panel kanan berisi kontrol untuk memuat file (TXT atau gambar), memilih mode algoritma, menampilkan statistik hasil pencarian (jumlah kasus dan waktu), serta mengekspor solusi. MainWindow mengelola siklus hidup Solver yang berjalan di QThread terpisah, menghubungkan sinyal progress() dan finished() dari Solver ke slot yang memperbarui tampilan, dan menyediakan mekanisme pembatalan (cancel) pencarian yang sedang berjalan.

MainWindow.h

```
#pragma once
#include "BoardWidget.h"
#include "Solver.h"
#include "Puzzle.h"
#include <QMainWindow>
#include <QThread>
class QLabel;
class QPushButton;
class QCheckBox;
class MainWindow : public QMainWindow {
   Q_OBJECT
public:
   explicit MainWindow(QWidget* parent = nullptr);
   ~MainWindow();
private slots:
   void onOpenTxt();
   void onOpenImage();
   void onSolve();
   void onCancel();
   void onExportImage();
   void onExportTxt();
   void onProgress(Puzzle snap, long long cases);
   void onFinished(Puzzle result, SolveStats stats, QString msg);
   void onOptimizedToggled(bool checked);
private:
   void setUiEnabled(bool enabled);
   BoardWidget* m_board = nullptr;
   QLabel* m_status = nullptr;
   QLabel* m_statsCases = nullptr;
   QLabel* m_statsTime = nullptr;
   QPushButton* m_btnSolve = nullptr;
   QPushButton* m_btnOpenTxt = nullptr;
   QPushButton* m_btnOpenImg = nullptr;
   QPushButton* m_btnExportTxt = nullptr;
   QPushButton* m_btnExportImg = nullptr;
   QCheckBox* m_chkOptimized = nullptr;
   Puzzle m_current;
   SolveStats m_lastStats;
   QThread m_workerThread;
   Solver* m_solver = nullptr;
   bool m_solving = false;
};
```

MainWindow.cpp

```
#include "MainWindow.h"
#include "ImageIO.h"
#include <QFileDialog>
#include <QLabel>
#include <QPushButton>
#include <QCheckBox>
#include <QVBoxLayout>
#include <QHBoxLayout>
#include <QFrame>
#include <QMessageBox>
#include <QStatusBar>
#include <QWidget>
#include <QMetaObject>
MainWindow::MainWindow(QWidget* parent) : QMainWindow(parent) {
   setWindowTitle("Queens Solver - Tucil1_13524017");
   setMinimumSize(1000, 660);
   auto* central = new QWidget(this);
   central->setObjectName("centralBg");
   auto* rootV = new QVBoxLayout(central);
   rootV->setContentsMargins(24, 16, 24, 16);
   rootV->setSpacing(12);
   auto* header = new QLabel("Queens Solver", central);
   header->setObjectName("headerTitle");
   header->setFixedHeight(48);
   rootV->addWidget(header);
   auto* card = new QFrame(central);
   card->setObjectName("mainCard");
   auto* bodyH = new QHBoxLayout(card);
   bodyH->setContentsMargins(20, 20, 20, 20);
   bodyH->setSpacing(24);
   m_board = new BoardWidget(card);
   m_board->setMinimumSize(480, 480);
   bodyH->addWidget(m_board, 1);
   auto* panel = new QWidget(card);
   panel->setFixedWidth(340);
   auto* pv = new QVBoxLayout(panel);
   pv->setContentsMargins(8, 8, 8, 8);
   pv->setSpacing(20);
   auto* lblUpload = new QLabel("Unggah file untuk dicarikan solusi",
panel);
   lblUpload->setObjectName("sectionTitle");
   pv->addWidget(lblUpload);
   auto* hUpload = new QHBoxLayout();
   hUpload->setSpacing(10);
   m_btnOpenTxt = new QPushButton("File .txt", panel);
   m_btnOpenTxt->setObjectName("outlineBtn");
   m_btnOpenImg = new QPushButton("File image", panel);
   m_btnOpenImg->setObjectName("outlineBtn");
```

```
    hUpload->addWidget(m_btnOpenTxt);
    hUpload->addWidget(m_btnOpenImg);
    pv->addLayout(hUpload);
    auto* sep1 = new QFrame(panel);
    sep1->setFrameShape(QFrame::HLine);
    sep1->setObjectName("separator");
    pv->addWidget(sep1);
    auto* lblSolve = new QLabel("Cari solusi permasalahan", panel);
    lblSolve->setObjectName("sectionTitle");
    pv->addWidget(lblSolve);
    m_chkOptimized = new QCheckBox("Optimasi", panel);
    m_chkOptimized->setChecked(false);
    pv->addWidget(m_chkOptimized);
    m_btnSolve = new QPushButton("Metode Brute Force", panel);
    m_btnSolve->setObjectName("solveBtn");
    m_btnSolve->setFixedHeight(44);
    m_btnSolve->setEnabled(false);
    pv->addWidget(m_btnSolve);
    auto* cardStats = new QFrame(panel);
    cardStats->setObjectName("statsCard");
    auto* statsV = new QVBoxLayout(cardStats);
    statsV->setContentsMargins(14, 16, 14, 16);
    statsV->setSpacing(10);
    auto* lblHasil = new QLabel("Hasil pencarian solusi", cardStats);
    lblHasil->setObjectName("statsTitle");
    statsV->addWidget(lblHasil);
    m_statsCases = new QLabel("• Banyak percobaan : -", cardStats);
    m_statsCases->setObjectName("statsItem");
    statsV->addWidget(m_statsCases);
    m_statsTime = new QLabel("• Waktu pencarian : -", cardStats);
    m_statsTime->setObjectName("statsItem");
    statsV->addWidget(m_statsTime);
    pv->addWidget(cardStats);
    auto* sep2 = new QFrame(panel);
    sep2->setFrameShape(QFrame::HLine);
    sep2->setObjectName("separator");
    pv->addWidget(sep2);
    auto* lblSave = new QLabel("Simpan hasil pencarian", panel);
    lblSave->setObjectName("sectionTitle");
    pv->addWidget(lblSave);
    auto* hSave = new QHBoxLayout();
    hSave->setSpacing(10);
    m_btnExportTxt = new QPushButton("File .txt", panel);
    m_btnExportTxt->setObjectName("outlineBtn");
    m_btnExportImg = new QPushButton("File image", panel);
    m_btnExportImg->setObjectName("outlineBtn");
    hSave->addWidget(m_btnExportTxt);
    hSave->addWidget(m_btnExportImg);
```

```
   pv->addLayout(hSave);
   pv->addStretch(1);
   m_status = new QLabel("Load puzzle dulu.", panel);
   m_status->setObjectName("statusLabel");
   m_status->setWordWrap(true);
   pv->addWidget(m_status);
   bodyH->addWidget(panel);
   rootV->addWidget(card, 1);
   setCentralWidget(central);
   connect(m_btnOpenTxt, &QPushButton::clicked, this,
&MainWindow::onOpenTxt);
   connect(m_btnOpenImg, &QPushButton::clicked, this,
&MainWindow::onOpenImage);
   connect(m_btnSolve,   &QPushButton::clicked, this,
&MainWindow::onSolve);
   connect(m_btnExportTxt, &QPushButton::clicked, this,
&MainWindow::onExportTxt);
   connect(m_btnExportImg, &QPushButton::clicked, this,
&MainWindow::onExportImage);
   connect(m_chkOptimized, &QCheckBox::toggled,  this,
&MainWindow::onOptimizedToggled);
   m_solver = new Solver();
   m_solver->moveToThread(&m_workerThread);
   connect(&m_workerThread, &QThread::finished, m_solver,
&QObject::deleteLater);
   connect(m_solver, &Solver::progress, this, &MainWindow::onProgress,
Qt::QueuedConnection);
   connect(m_solver, &Solver::finished, this, &MainWindow::onFinished,
Qt::QueuedConnection);
   m_workerThread.start();
   onOptimizedToggled(m_chkOptimized->isChecked());
   setStyleSheet(R"(
       QWidget#centralBg {
           background-color: #FFB3C6;
       }
       QFrame#mainCard {
           background-color: white;
           border-radius: 16px;
       }
       QLabel#headerTitle {
           font-size: 26px;
           font-weight: bold;
           color: #1a1a1a;
           background-color: transparent;
       }
       QLabel#sectionTitle {
           font-size: 16px;
           font-weight: bold;
```

```
            color: #C2415C;
            background-color: transparent;
        }
        QFrame#statsCard {
            background-color: #FFF0F3;
            border-radius: 10px;
            border: 1px solid #FFB3C6;
        }
        QLabel#statsTitle {
            font-size: 15px;
            font-weight: bold;
            color: #6D2B3D;
            background-color: transparent;
        }
        QLabel#statsItem {
            font-size: 14px;
            color: #6D2B3D;
            background-color: transparent;
        }
        QLabel#statusLabel {
            font-size: 13px;
            color: #6D2B3D;
            background-color: #FFF0F3;
            border-radius: 8px;
            padding: 10px;
        }
        QPushButton#outlineBtn {
            background-color: white;
            color: #C2415C;
            border: 1.5px solid #FFB3C6;
            border-radius: 20px;
            padding: 8px 16px;
            font-size: 14px;
        }
        QPushButton#outlineBtn:hover {
            background-color: #FFF0F3;
            border-color: #FF85A1;
        }
        QPushButton#outlineBtn:pressed {
            background-color: #FFD6E0;
        }
        QPushButton#solveBtn {
            background-color: #FF85A1;
            color: white;
            border: none;
            border-radius: 22px;
            font-size: 15px;
            font-weight: bold;
```

```
        }
        QPushButton#solveBtn:hover {
            background-color: #E05C7A;
        }
        QPushButton#solveBtn:pressed {
            background-color: #C2415C;
        }
        QPushButton#solveBtn:disabled {
            background-color: #FFCCD5;
            color: #D4A0AC;
        }
        QCheckBox {
            color: #6D2B3D;
            font-size: 14px;
            spacing: 8px;
            background-color: transparent;
        }
        QCheckBox::indicator {
            width: 18px;
            height: 18px;
            border-radius: 4px;
            border: 2px solid #FFB3C6;
            background-color: white;
        }
        QCheckBox::indicator:checked {
            background-color: #FF85A1;
            border-color: #FF85A1;
        }
        QFrame#separator {
            color: #FFD6E0;
            background-color: #FFD6E0;
            max-height: 1px;
        }
        QStatusBar {
            background-color: #FFB3C6;
            color: #6D2B3D;
            font-size: 12px;
        }
    )");
}
MainWindow::~MainWindow() {
    if (m_solver) {
        QMetaObject::invokeMethod(m_solver, "cancel",
Qt::QueuedConnection);
    }
    m_workerThread.quit();
    m_workerThread.wait();
}
```

```
void MainWindow::setUiEnabled(bool enabled) {
   m_btnOpenTxt->setEnabled(enabled);
   m_btnOpenImg->setEnabled(enabled);
   m_chkOptimized->setEnabled(enabled);
   m_btnExportTxt->setEnabled(enabled);
   m_btnExportImg->setEnabled(enabled);
   m_solving = !enabled;
   if (!enabled) {
       m_btnSolve->setText("Batalkan pencarian");
       m_btnSolve->setObjectName("cancelBtn");
       m_btnSolve->setStyleSheet(
           "QPushButton { background-color: #6D2B3D; color: white;
border: none;"
           "border-radius: 22px; font-size: 14px; font-weight: bold; }"
           "QPushButton:hover { background-color: #8B3A50; }"
       );
   } else {
       m_btnSolve->setText("Metode Brute Force");
       m_btnSolve->setObjectName("solveBtn");
       m_btnSolve->setStyleSheet("");
       m_btnSolve->setEnabled(m_current.isValid());
   }
}
void MainWindow::onOptimizedToggled(bool checked) {
   if (!m_solver) return;
   QMetaObject::invokeMethod(m_solver, "setOptimized",
Qt::QueuedConnection,
                             Q_ARG(bool, checked));
}
void MainWindow::onOpenTxt() {
   const QString path = QFileDialog::getOpenFileName(
       this, "Open Puzzle (.txt)", "", "Text files (*.txt)");
   if (path.isEmpty()) return;
   QString err;
   Puzzle pz;
   if (!ImageIO::loadFromTxt(path, pz, &err)) {
       QMessageBox::warning(this, "Error", err);
       return;
   }
   m_current = pz;
   m_board->setPuzzle(m_current);
   m_status->setText("File dimuat: " + path);
   m_statsCases->setText("• Banyak percobaan : -");
   m_statsTime->setText("• Waktu pencarian : -");
   m_btnSolve->setEnabled(true);
}
void MainWindow::onOpenImage() {
   const QString path = QFileDialog::getOpenFileName(
```

```
        this, "Open Puzzle Image", "", "Images (*.png *.jpg *.jpeg)");
    if (path.isEmpty()) return;
    QString err;
    Puzzle pz;
    if (!ImageIO::loadFromImage(path, pz, &err)) {
        QMessageBox::information(this, "Info", err);
        return;
    }
    m_current = pz;
    m_board->setPuzzle(m_current);
    m_status->setText("File dimuat: " + path);
    m_statsCases->setText("• Banyak percobaan : -");
    m_statsTime->setText("• Waktu pencarian : -");
    m_btnSolve->setEnabled(true);
}
void MainWindow::onSolve() {
    if (m_solving) {
        onCancel();
        return;
    }
    if (!m_current.isValid()) return;
    setUiEnabled(false);
    m_status->setText("Mencari solusi...");
    m_statsCases->setText("• Banyak percobaan : 0");
    m_statsTime->setText("• Waktu pencarian : ...");
    QMetaObject::invokeMethod(m_solver, "setOptimized",
Qt::QueuedConnection,
                              Q_ARG(bool, m_chkOptimized->isChecked()));
    QMetaObject::invokeMethod(m_solver, "solve", Qt::QueuedConnection,
                              Q_ARG(Puzzle, m_current));
}
void MainWindow::onCancel() {
    m_status->setText("Membatalkan...");
    if (m_solver) {
        m_solver->cancel();
    }
}
void MainWindow::onExportTxt() {
    if (!m_current.isValid()) {
        QMessageBox::information(this, "Info", "Belum ada solusi.");
        return;
    }
    QString path = QFileDialog::getSaveFileName(
        this, "Save Solution TXT", "solution.txt", "Text files (*.txt)");
    if (path.isEmpty()) return;
    QString err;
    if (!ImageIO::saveSolutionTxt(path, m_board->puzzle(), m_lastStats,
&err)) {
```

```
        QMessageBox::warning(this, "Error", err);
        return;
    }
    m_status->setText("Tersimpan: " + path);
}
void MainWindow::onExportImage() {
    if (!m_current.isValid()) {
        QMessageBox::information(this, "Info", "Belum ada puzzle.");
        return;
    }
    const QString path = QFileDialog::getSaveFileName(
        this, "Save Solution Image", "solution.png", "PNG (*.png)");
    if (path.isEmpty()) return;
    QString err;
    if (!ImageIO::saveSolutionImage(path, m_board->puzzle(), &err)) {
        QMessageBox::warning(this, "Error", err);
        return;
    }
    m_status->setText("Tersimpan: " + path);
}
void MainWindow::onProgress(Puzzle snap, long long cases) {
    m_board->setPuzzle(std::move(snap));
    m_statsCases->setText(QString("• Banyak percobaan : %1").arg(cases));
    m_statsTime->setText("• Waktu pencarian : ...");
}
void MainWindow::onFinished(Puzzle result, SolveStats stats, QString
msg) {
    setUiEnabled(true);
    m_board->setPuzzle(result);
    m_current = result;
    m_lastStats = stats;
    m_statsCases->setText(QString("• Banyak percobaan :
%1").arg(stats.casesChecked));
    m_statsTime->setText(QString("• Waktu pencarian : %1
ms").arg(stats.elapsedMs));
    m_status->setText(msg);
}
```

3. BoardWidget.h dan BoardWidget.cpp

   File ini mendefinisikan widget kustom Qt yang bertanggung jawab untuk merender papan secara visual. Pada setiap pemanggilan paintEvent(), widget menggambar setiap cell dengan warna region-nya menggunakan QPainter, menambahkan garis grid hitam sebagai pembatas, dan mencetak simbol queen (♛) berwarna emas pada cell yang terisi. Ukuran cell dihitung secara dinamis berdasarkan ukuran widget agar papan selalu fit di layar. Widget ini menerima

update state melalui metode setPuzzle(), yang dipanggil secara berkala selama proses solving untuk menampilkan progres secara real-time.

BoardWidget.h

```
#pragma once
#include "Puzzle.h"
#include <QWidget>
class BoardWidget : public QWidget {
   Q_OBJECT
public:
   explicit BoardWidget(QWidget* parent = nullptr);
   void setPuzzle(const Puzzle& pz);
   const Puzzle& puzzle() const { return m_pz; }
   QSize minimumSizeHint() const override;
protected:
   void paintEvent(QPaintEvent* e) override;
private:
   Puzzle m_pz;
};
```

BoardWidget.cpp

```
#include "BoardWidget.h"
#include <QPainter>
BoardWidget::BoardWidget(QWidget* parent) : QWidget(parent) {
   setAutoFillBackground(true);
}
void BoardWidget::setPuzzle(const Puzzle& pz) {
   m_pz = pz;
   update();
}
QSize BoardWidget::minimumSizeHint() const {
   return QSize(520, 520);
}
void BoardWidget::paintEvent(QPaintEvent*) {
   QPainter p(this);
   p.setRenderHint(QPainter::Antialiasing, true);
   p.fillRect(rect(), Qt::white);
   if (!m_pz.isValid()) {
       p.setPen(Qt::black);
       p.drawText(rect(), Qt::AlignCenter, "Belum ada puzzle / puzzle
invalid");
       return;
   }
   int n = m_pz.n;
   int pad = 10;
```

```
    int availW = width() - 2*pad;
    int availH = height() - 2*pad;
    int cell = std::max(10, std::min(availW, availH) / n);
    int boardW = n * cell;
    int boardH = n * cell;
    int ox = (width() - boardW) / 2;
    int oy = (height() - boardH) / 2;
    for (int r = 0; r < n; r++) {
        for (int c = 0; c < n; c++) {
            int id = m_pz.regionId[r][c];
            QColor col = (id >= 0 && id < (int)m_pz.regionColors.size())
                ? m_pz.regionColors[id] : QColor("#E5E7EB");

            QRect rect(ox + c*cell, oy + r*cell, cell, cell);
            p.fillRect(rect, col);
            p.setPen(QPen(Qt::black, 1));
            p.drawRect(rect);
            if (m_pz.queen[r][c]) {
                p.setPen(QPen(QColor("#FFD700"), 1));
                p.setBrush(QColor("#FFD700"));
                QFont f = p.font();
                f.setPixelSize(cell * 0.6);
                f.setBold(true);
                p.setFont(f);
                p.drawText(rect, Qt::AlignCenter, "♛");
                p.setBrush(Qt::NoBrush);
            }
        }
    }
}
```

4. Puzzle.h dan Puzzle.cpp

   Fini mendefinisikan struktur data inti yang merepresentasikan state papan permainan. Struct Puzzle menyimpan ukuran papan n, matriks regionId yang memetakan setiap sel ke ID region-nya, matriks queen yang mencatat posisi queen yang sudah ditempatkan, serta array regionColors untuk keperluan rendering. Selain itu, file ini menyediakan metode isValid() untuk memverifikasi integritas data puzzle, clearQueens() untuk mereset penempatan queen sebelum proses solving dimulai, dan dua metode bantu toAscii() serta toAsciiOverlay() untuk merepresentasikan papan dalam format teks.

   Puzzle.h

```
#pragma once
```

```
#include <vector>
#include <QString>
#include <QColor>
#include <QMetaType>
struct Puzzle {
   int n = 0;
   std::vector<std::vector<int>> regionId;
   std::vector<std::vector<bool>> queen;
   std::vector<QColor> regionColors;
   bool isValid() const;
   void clearQueens();
   QString toAscii(char queenChar = '#', char emptyChar = '.') const;
   QString toAsciiOverlay(char queenChar = '#') const;
};
Q_DECLARE_METATYPE(Puzzle)
```

Puzzle.cpp

```
#include "Puzzle.h"
#include <unordered_set>
bool Puzzle::isValid() const {
   if (n <= 0) return false;
   if ((int)regionId.size() != n) return false;
   for (const auto& row : regionId) {
       if ((int)row.size() != n) return false;
   }
   if ((int)queen.size() != n) return false;
   for (const auto& row : queen) {
       if ((int)row.size() != n) return false;
   }
   std::unordered_set<int> uniq;
   uniq.reserve((size_t)n);
   int maxId = -1;
   for (int r = 0; r < n; r++) {
       for (int c = 0; c < n; c++) {
           const int id = regionId[r][c];
           if (id < 0) return false;
           uniq.insert(id);
           if (id > maxId) maxId = id;
       }
   }
   if ((int)uniq.size() != n) return false;
   if (n > 26) return false; // max A-Z
   if (!regionColors.empty()) {
       if ((int)regionColors.size() != n) return false;
   }
   // pastikan semua id ada di range [0..n-1]
   for (int r = 0; r < n; r++) {
```

```
        for (int c = 0; c < n; c++) {
            const int id = regionId[r][c];
            if (id < 0 || id >= n) return false;
        }
    }
    return true;
}
void Puzzle::clearQueens() {
    queen.assign(n, std::vector<bool>(n, false));
}
QString Puzzle::toAscii(char queenChar, char emptyChar) const {
    QString out;
    for (int r = 0; r < n; r++) {
        for (int c = 0; c < n; c++) {
            out += (queen[r][c] ? queenChar : emptyChar);
        }
        out += "\n";
    }
    return out;
}
QString Puzzle::toAsciiOverlay(char queenChar) const {
    QString out;
    if (n <= 0) return out;
    for (int r = 0; r < n; r++) {
        for (int c = 0; c < n; c++) {
            if (queen.size() == (size_t)n && queen[r].size() == (size_t)n
&& queen[r][c]) {
                out += queenChar;
            } else {
                int id = 0;
                if (regionId.size() == (size_t)n && regionId[r].size() ==
(size_t)n) {
                    id = regionId[r][c];
                }
                out += (id >= 0 && id < 26) ? QChar('A' + id) :
QChar('?');
            }
        }
        out += "\n";
    }
    return out;
}
```

5. Solver.h dan Solver.cpp

   File ini adalah inti algoritmik program. Kelas Solver mewarisi QObject sehingga dapat dijalankan di thread terpisah menggunakan QThread, menjaga antarmuka tetap responsif selama proses pencarian berlangsung. Di dalamnya terdapat dua

implementasi algoritma: dfsPure() yang menjalankan brute force murni tanpa pemangkasan dan hanya memvalidasi konfigurasi di akhir via isCompleteValid(), serta dfsOptimized() yang menggunakan backtracking dengan pruning melalui fungsi canPlace() untuk memangkas cabang tidak valid sedini mungkin. Dua flag atomik m_cancel dan m_optimized digunakan untuk mengontrol eksekusi secara thread-safe. Solver mengemit sinyal progress() setiap 2.000 kasus dan finished() ketika pencarian selesai.

Solver.h

```
#pragma once
#include "Puzzle.h"
#include <QObject>
#include <atomic>
#include <chrono>
#include <QMetaType>
#include <QString>
struct SolveStats {
   long long casesChecked = 0;
   int elapsedMs = 0;
   bool found = false;
};
Q_DECLARE_METATYPE(SolveStats)
class Solver : public QObject {
   Q_OBJECT
public:
   explicit Solver(QObject* parent = nullptr);
public slots:
   void solve(Puzzle pz);
   void cancel();
   void setOptimized(bool on);
signals:
   void progress(Puzzle snapshot, long long casesChecked);
   void finished(Puzzle result, SolveStats stats, QString message);
private:
   std::atomic<bool> m_cancel{false};
   std::atomic<bool> m_optimized{false};
   bool dfsPure(Puzzle& pz, int row, SolveStats& st, int emitEvery);
   bool dfsOptimized(Puzzle& pz, int row, SolveStats& st, int
emitEvery);
   bool isCompleteValid(const Puzzle& pz) const;
   bool canPlace(const Puzzle& pz, int r, int c) const;
};
```

Solver.cpp

```
#include "Solver.h"
#include <QThread>
#include <vector>
Solver::Solver(QObject* parent) : QObject(parent) {}
void Solver::cancel() {
   m_cancel.store(true);
}
void Solver::setOptimized(bool on) {
   m_optimized.store(on);
}
void Solver::solve(Puzzle pz) {
   m_cancel.store(false);
   SolveStats st;
   if (!pz.isValid()) {
       emit finished(pz, st, "Puzzle invalid.");
       return;
   }
   pz.clearQueens();
   auto start = std::chrono::steady_clock::now();
   const int emitEvery = 2000;
   bool ok = m_optimized.load() ? dfsOptimized(pz, 0, st, emitEvery)
                                : dfsPure(pz, 0, st, emitEvery);
   auto end = std::chrono::steady_clock::now();
   st.elapsedMs =
(int)std::chrono::duration_cast<std::chrono::milliseconds>(end -
start).count();
   st.found = ok;
   QString msg = ok ? "Solusi ditemukan."
                    : (m_cancel.load() ? "Dibatalkan." : "Tidak ada
solusi.");
   emit finished(pz, st, msg);
}
// Brute force murni
bool Solver::dfsPure(Puzzle& pz, int row, SolveStats& st, int emitEvery)
{
   if (m_cancel.load()) return false;
   if (row == pz.n) {
       return isCompleteValid(pz);
   }
   for (int c = 0; c < pz.n; c++) {
       if (m_cancel.load()) return false;
       st.casesChecked++;
       // reset baris ini, lalu taruh queen di kolom c
       for (int k = 0; k < pz.n; k++)
           pz.queen[row][k] = false;
       pz.queen[row][c] = true;
       if (emitEvery > 0 && (st.casesChecked % emitEvery == 0)) {
```

```
            emit progress(pz, st.casesChecked);
            QThread::yieldCurrentThread();
        }
        if (dfsPure(pz, row + 1, st, emitEvery))
            return true;
        pz.queen[row][c] = false;
    }
    return false;
}
// Versi dengan pruning untuk skip posisi yang jelas invalid
bool Solver::dfsOptimized(Puzzle& pz, int row, SolveStats& st, int
emitEvery) {
    if (m_cancel.load()) return false;
    if (row == pz.n) return true;
    for (int c = 0; c < pz.n; c++) {
        if (m_cancel.load()) return false;
        st.casesChecked++;
        if (canPlace(pz, row, c)) {
            pz.queen[row][c] = true;
            if (emitEvery > 0 && (st.casesChecked % emitEvery == 0)) {
                emit progress(pz, st.casesChecked);
                QThread::yieldCurrentThread();
            }
            if (dfsOptimized(pz, row + 1, st, emitEvery))
                return true;
            pz.queen[row][c] = false;
        }
    }
    return false;
}
bool Solver::isCompleteValid(const Puzzle& pz) const {
    const int n = pz.n;
    std::vector<int> colCount(n, 0);
    std::vector<int> regCount(n, 0);
    for (int r = 0; r < n; r++) {
        int rowQueens = 0;
        int placedCol = -1;
        for (int c = 0; c < n; c++) {
            if (pz.queen[r][c]) {
                rowQueens++;
                placedCol = c;
            }
        }
        if (rowQueens != 1) return false;
        colCount[placedCol]++;
        if (colCount[placedCol] > 1) return false;
        const int reg = pz.regionId[r][placedCol];
        regCount[reg]++;
```

```
        if (regCount[reg] > 1) return false;
        // cek apakah ada queen tetangga (8 arah)
        for (int dr = -1; dr <= 1; dr++) {
            for (int dc = -1; dc <= 1; dc++) {
                if (dr == 0 && dc == 0) continue;
                const int nr = r + dr;
                const int nc = placedCol + dc;
                if (nr >= 0 && nr < n && nc >= 0 && nc < n) {
                    if (pz.queen[nr][nc]) return false;
                }
            }
        }
    }
    return true;
}
bool Solver::canPlace(const Puzzle& pz, int r, int c) const {
    // cek kolom
    for (int rr = 0; rr < pz.n; rr++) {
        if (pz.queen[rr][c]) return false;
    }
    // cek tetangga 8 arah
    for (int dr = -1; dr <= 1; dr++) {
        for (int dc = -1; dc <= 1; dc++) {
            if (dr == 0 && dc == 0) continue;
            int nr = r + dr, nc = c + dc;
            if (nr >= 0 && nr < pz.n && nc >= 0 && nc < pz.n) {
                if (pz.queen[nr][nc]) return false;
            }
        }
    }
    // cek region
    int myRegion = pz.regionId[r][c];
    for (int rr = 0; rr < pz.n; rr++) {
        for (int cc = 0; cc < pz.n; cc++) {
            if (pz.regionId[rr][cc] == myRegion && pz.queen[rr][cc])
                return false;
        }
    }
    return true;
}
```

6. ImageIO.h dan ImageIO.cpp

   File ini menangani semua operasi input dan output file. Fungsi loadFromTxt() mem-parsing file teks berformat NxN huruf kapital A–Z menjadi objek Puzzle, lengkap dengan validasi jumlah region dan karakter. Fungsi loadFromImage() melakukan hal serupa dari file gambar dengan cara mendeteksi garis grid secara

otomatis (menganalisis baris dan kolom dengan lebih dari 90% piksel gelap), lalu menyampling warna di titik tengah setiap cell untuk menentukan region-nya. Dua fungsi ekspor, saveSolutionTxt() dan saveSolutionImage(), menyimpan hasil solving beserta statistiknya masing-masing ke format teks dan gambar PNG.

ImageIO.h

```
#pragma once
#include "Puzzle.h"
#include "Solver.h"
#include <QString>
namespace ImageIO {
   bool loadFromTxt(const QString& path, Puzzle& out, QString* err =
nullptr);
   bool loadFromImage(const QString& path, Puzzle& out, QString* err =
nullptr);
   bool saveSolutionTxt(const QString& path, const Puzzle& pz, const
SolveStats& stats, QString* err = nullptr);
   bool saveSolutionImage(const QString& path, const Puzzle& pz,
QString* err = nullptr);
}
```

ImageIO.cpp

```
#include "ImageIO.h"
#include <QFile>
#include <QTextStream>
#include <QImage>
#include <QPainter>
#include <unordered_map>
static QColor niceColor(int i) {
   static std::vector<QColor> palette = {
       QColor("#94A3B8"), QColor("#6EE7B7"), QColor("#93C5FD"),
QColor("#F9A8D4"),
       QColor("#C4B5FD"), QColor("#FCA5A5"), QColor("#67E8F9"),
QColor("#86EFAC"),
       QColor("#FF6B6B"), QColor("#4ECDC4"), QColor("#45B7D1"),
QColor("#BB8FCE"),
       QColor("#F97316"), QColor("#98D8C8"), QColor("#A78BFA"),
QColor("#34D399"),
       QColor("#FB7185"), QColor("#38BDF8"), QColor("#A3E635"),
QColor("#E879F9"),
       QColor("#2DD4BF"), QColor("#457B9D"), QColor("#1D3557"),
QColor("#166534"),
       QColor("#7C3AED"), QColor("#9F1239")
   };
```

```
    return palette[i % (int)palette.size()];
}
bool ImageIO::loadFromTxt(const QString& path, Puzzle& out, QString*
err) {
    QFile f(path);
    if (!f.open(QIODevice::ReadOnly | QIODevice::Text)) {
        if (err) *err = "Gagal membuka file.";
        return false;
    }
    QTextStream in(&f);
    std::vector<QString> lines;
    while (!in.atEnd()) {
        const QString line = in.readLine().trimmed();
        if (!line.isEmpty()) lines.push_back(line);
    }
    if (lines.empty()) {
        if (err) *err = "File kosong.";
        return false;
    }
    const int n = (int)lines.size();
    if (n > 26) {
        if (err) *err = "Ukuran papan terlalu besar. Maksimum 26x26
(A-Z).";
        return false;
    }
    for (const auto& ln : lines) {
        if (ln.size() != n) {
            if (err) *err = "Input harus papan NxN (jumlah kolom = jumlah
baris).";
            return false;
        }
    }
    std::unordered_map<char, int> mp;
    mp.reserve((size_t)n);
    int nextId = 0;
    out.n = n;
    out.regionId.assign(n, std::vector<int>(n, 0));
    out.queen.assign(n, std::vector<bool>(n, false));
    for (int r = 0; r < n; r++) {
        for (int c = 0; c < n; c++) {
            const QChar qc = lines[r].at(c);
            const char ch = qc.toLatin1();
            if (ch < 'A' || ch > 'Z') {
                if (err) {
                    *err = QString("Karakter tidak valid di baris %1
kolom %2. "
                                   "Hanya boleh huruf A-Z.")
                               .arg(r + 1).arg(c + 1);
```

```
                }
                return false;
            }
            auto it = mp.find(ch);
            if (it == mp.end()) {
                mp.emplace(ch, nextId);
                out.regionId[r][c] = nextId;
                nextId++;
            } else {
                out.regionId[r][c] = it->second;
            }
        }
    }
    if (nextId != n) {
        if (err) {
            *err = QString("Jumlah region tidak valid. Ditemukan %1
region unik, "
                           "harus tepat %2 (sesuai ukuran papan).")
                       .arg(nextId).arg(n);
        }
        return false;
    }
    out.regionColors.resize(n);
    for (int i = 0; i < n; i++)
        out.regionColors[i] = niceColor(i);
    if (!out.isValid()) {
        if (err) *err = "Puzzle invalid setelah parsing (cek format
input).";
        return false;
    }
    return true;
}
bool ImageIO::loadFromImage(const QString& path, Puzzle& out, QString*
err) {
    QImage img(path);
    if (img.isNull()) {
        if (err) *err = "Gagal membaca gambar.";
        return false;
    }
    img = img.convertToFormat(QImage::Format_ARGB32);
    const int W = img.width();
    const int H = img.height();
    if (W <= 0 || H <= 0) {
        if (err) *err = "Ukuran gambar tidak valid.";
        return false;
    }
    if (W != H) {
        if (err) *err = "Gambar harus persegi (width == height) untuk
```

```
di-parse sebagai papan NxN.";
        return false;
    }
    auto isDark = [](QRgb px) -> bool {
        const int r = qRed(px), g = qGreen(px), b = qBlue(px);
        return (r + g + b) / 3 < 40;
    };
    std::vector<int> hLines;
    hLines.reserve(H);
    for (int y = 0; y < H; y++) {
        int darkCount = 0;
        for (int x = 0; x < W; x++) {
            if (isDark(img.pixel(x, y))) darkCount++;
        }
        const double frac = (double)darkCount / (double)W;
        if (frac > 0.90) hLines.push_back(y);
    }
    std::vector<int> vLines;
    vLines.reserve(W);
    for (int x = 0; x < W; x++) {
        int darkCount = 0;
        for (int y = 0; y < H; y++) {
            if (isDark(img.pixel(x, y))) darkCount++;
        }
        const double frac = (double)darkCount / (double)H;
        if (frac > 0.90) vLines.push_back(x);
    }
    auto compressLines = [](const std::vector<int>& lines) {
        std::vector<int> reps;
        for (int i = 0; i < (int)lines.size(); ) {
            int j = i;
            while (j + 1 < (int)lines.size() && lines[j + 1] == lines[j]
+ 1) j++;
            reps.push_back(lines[(i + j) / 2]);
            i = j + 1;
        }
        return reps;
    };
    const std::vector<int> h = compressLines(hLines);
    const std::vector<int> v = compressLines(vLines);
    std::vector<int> yCuts;
    std::vector<int> xCuts;
    if (!h.empty() && !v.empty()) {
        yCuts.clear();
        xCuts.clear();
        yCuts.push_back(-1);
        for (int yy : h) yCuts.push_back(yy);
        yCuts.push_back(H);
```

```
        xCuts.push_back(-1);
        for (int xx : v) xCuts.push_back(xx);
        xCuts.push_back(W);
    }
    int n = 0;
    if (!yCuts.empty() && !xCuts.empty()) {
        n = (int)yCuts.size() - 1;
        const int nx = (int)xCuts.size() - 1;
        if (n != nx) {
            if (err) *err = "Deteksi grid tidak konsisten (jumlah cell
baris != kolom).";
            return false;
        }
        int validRows = 0;
        for (int i = 0; i + 1 < (int)yCuts.size(); i++) {
            int top = yCuts[i] + 1;
            int bot = yCuts[i + 1] - 1;
            if (top <= bot) validRows++;
        }
        int validCols = 0;
        for (int i = 0; i + 1 < (int)xCuts.size(); i++) {
            int left = xCuts[i] + 1;
            int right = xCuts[i + 1] - 1;
            if (left <= right) validCols++;
        }
        if (validRows != validCols) {
            if (err) *err = "Deteksi cell gagal (area cell terlalu tipis
/ garis terlalu rapat).";
            return false;
        }
        n = validRows;
    } else {
        n = W;
    }
    if (n <= 0) {
        if (err) *err = "Gagal menentukan ukuran papan dari gambar.";
        return false;
    }
    if (n > 26) {
        if (err) *err = "Ukuran papan dari gambar terlalu besar. Maksimum
26x26 (A-Z).";
        return false;
    }
    out.n = n;
    out.regionId.assign(n, std::vector<int>(n, 0));
    out.queen.assign(n, std::vector<bool>(n, false));
    std::unordered_map<QRgb, int> colorToId;
    colorToId.reserve((size_t)n);
```

```
    std::vector<QColor> regionColorList;
    regionColorList.reserve((size_t)n);
    auto addColor = [&](QRgb key) -> int {
        auto it = colorToId.find(key);
        if (it != colorToId.end()) return it->second;
        const int id = (int)colorToId.size();
        colorToId.emplace(key, id);
        regionColorList.push_back(QColor(key));
        return id;
    };
    if (!yCuts.empty() && !xCuts.empty()) {
        int rr = 0;
        for (int i = 0; i + 1 < (int)yCuts.size(); i++) {
            const int top = yCuts[i] + 1;
            const int bot = yCuts[i + 1] - 1;
            if (top > bot) continue;
            int cc = 0;
            for (int j = 0; j + 1 < (int)xCuts.size(); j++) {
                const int left = xCuts[j] + 1;
                const int right = xCuts[j + 1] - 1;
                if (left > right) continue;
                const int sy = (top + bot) / 2;
                const int sx = (left + right) / 2;
                const QRgb px = img.pixel(sx, sy);
                if (isDark(px)) {
                    if (err) *err = "Sampling warna cell kena garis grid
(terlalu gelap). Pastikan cell berisi warna solid.";
                    return false;
                }
                out.regionId[rr][cc] = addColor(px);
                cc++;
            }
            rr++;
        }
    } else {
        for (int r = 0; r < n; r++) {
            for (int c = 0; c < n; c++) {
                const QRgb px = img.pixel(c, r);
                if (isDark(px)) {
                    if (err) *err = "Terdeteksi pixel gelap (mirip garis
grid) pada mode fallback. "
                                    "Gunakan gambar dengan grid lines
yang jelas, atau gambar NxN pixel murni.";
                    return false;
                }
                out.regionId[r][c] = addColor(px);
            }
        }
```

```
    }
    const int regionCount = (int)colorToId.size();
    if (regionCount != n) {
        if (err) {
            *err = QString("Jumlah region dari gambar tidak valid.
Ditemukan %1 warna/region unik, "
                           "harus tepat %2 (sesuai ukuran papan).")
                       .arg(regionCount).arg(n);
        }
        return false;
    }
    out.regionColors = regionColorList;
    if (!out.isValid()) {
        if (err) *err = "Puzzle invalid setelah parsing dari gambar.";
        return false;
    }
    return true;
}
bool ImageIO::saveSolutionTxt(const QString& path, const Puzzle& pz,
const SolveStats& st, QString* err) {
    if (!pz.isValid()) {
        if (err) *err = "Puzzle tidak valid.";
        return false;
    }
    QFile f(path);
    if (!f.open(QIODevice::WriteOnly | QIODevice::Text)) {
        if (err) *err = "Gagal menyimpan file txt.";
        return false;
    }
    QTextStream out(&f);
    for (int r = 0; r < pz.n; r++) {
        for (int c = 0; c < pz.n; c++) {
            if (pz.queen[r][c]) {
                out << '#';
            } else {
                int id = pz.regionId[r][c];
                QChar ch = (id >= 0 && id < 26) ? QChar('A' + id) :
QChar('?');
                out << ch;
            }
        }
        out << "\n";
    }
    out << "\n";
    out << "Waktu pencarian: " << st.elapsedMs << " ms\n";
    out << "Banyak kasus yang ditinjau: " << st.casesChecked << "
kasus\n";
    return true;
```

```
}
bool ImageIO::saveSolutionImage(const QString& path, const Puzzle& pz,
QString* err) {
   if (!pz.isValid()) {
       if (err) *err = "Puzzle tidak valid.";
       return false;
   }
   const int cell = 60;
   const int pad = 10;
   const int W = pad*2 + pz.n * cell;
   const int H = pad*2 + pz.n * cell;
   QImage outImg(W, H, QImage::Format_ARGB32);
   outImg.fill(Qt::white);
   QPainter p(&outImg);
   p.setRenderHint(QPainter::Antialiasing, true);
   for (int r = 0; r < pz.n; r++) {
       for (int c = 0; c < pz.n; c++) {
           int id = pz.regionId[r][c];
           QColor col = (id >= 0 && id < (int)pz.regionColors.size())
               ? pz.regionColors[id] : QColor("#E5E7EB");
           QRect rect(pad + c*cell, pad + r*cell, cell, cell);
           p.fillRect(rect, col);
           p.setPen(QPen(Qt::black, 1));
           p.drawRect(rect);
           if (pz.queen[r][c]) {
               p.setPen(QPen(QColor("#FFD700"), 1));
               p.setBrush(QColor("#FFD700"));
               QFont f = p.font();
               f.setPixelSize(cell * 0.6);
               f.setBold(true);
               p.setFont(f);
               p.drawText(rect, Qt::AlignCenter, "♛");
               p.setBrush(Qt::NoBrush);
           }
       }
   }
   if (!outImg.save(path)) {
       if (err) *err = "Gagal menyimpan image.";
       return false;
   }
   return true;
}
```

7. txt2img.cpp

   File ini adalah program utilitas mandiri (standalone) yang mengonversi file puzzle format .txt menjadi gambar .png. Program ini mem-parsing file teks input dengan cara yang sama seperti ImageIO::loadFromTxt(), kemudian merender

papan berwarna menggunakan QPainter dengan garis grid hitam setebal 2 piksel dan cell berukuran 60×60 piksel. Keluarannya adalah gambar yang dapat langsung digunakan kembali sebagai input gambar untuk program utama, berguna untuk keperluan pengujian dan pembuatan test case.

```
#include <QApplication>
#include <QImage>
#include <QPainter>
#include <QFile>
#include <QTextStream>
#include <QColor>
#include <vector>
#include <unordered_map>
static QColor regionColor(int i) {
   static std::vector<QColor> palette = {
       QColor("#94A3B8"), QColor("#6EE7B7"), QColor("#93C5FD"),
QColor("#F9A8D4"),
       QColor("#C4B5FD"), QColor("#FCA5A5"), QColor("#67E8F9"),
QColor("#86EFAC"),
       QColor("#FF6B6B"), QColor("#4ECDC4"), QColor("#45B7D1"),
QColor("#BB8FCE"),
       QColor("#F97316"), QColor("#98D8C8"), QColor("#A78BFA"),
QColor("#34D399"),
       QColor("#FB7185"), QColor("#38BDF8"), QColor("#A3E635"),
QColor("#E879F9"),
       QColor("#2DD4BF"), QColor("#457B9D"), QColor("#1D3557"),
QColor("#166534"),
       QColor("#7C3AED"), QColor("#9F1239")
   };
   return palette[i % (int)palette.size()];
}
int main(int argc, char* argv[]) {
   QApplication app(argc, argv);
   if (argc < 3) {
       qWarning("Usage: txt2img <input.txt> <output.png>");
       return 1;
   }
   const QString inputPath  = argv[1];
   const QString outputPath = argv[2];
   QFile f(inputPath);
   if (!f.open(QIODevice::ReadOnly | QIODevice::Text)) {
       qWarning("Gagal membuka file: %s", qPrintable(inputPath));
       return 1;
   }
   QTextStream in(&f);
   std::vector<QString> lines;
```

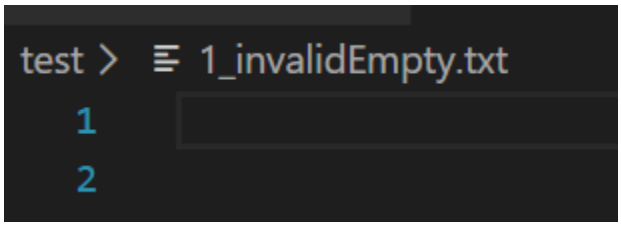
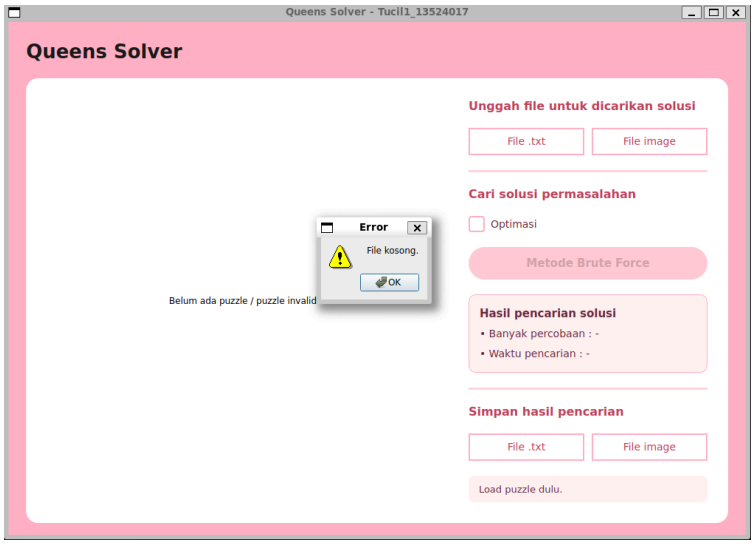
```
   while (!in.atEnd()) {
       QString line = in.readLine().trimmed();
       if (!line.isEmpty()) lines.push_back(line);
   }
   if (lines.empty()) {
       qWarning("File kosong.");
       return 1;
   }
   const int n = (int)lines.size();
   for (const auto& ln : lines) {
       if (ln.size() != n) {
           qWarning("Input harus NxN.");
           return 1;
       }
   }
   std::unordered_map<char, int> mp;
   int nextId = 0;
   std::vector<std::vector<int>> regionId(n, std::vector<int>(n, 0));
   for (int r = 0; r < n; r++) {
       for (int c = 0; c < n; c++) {
           char ch = lines[r].at(c).toLatin1();
           if (ch < 'A' || ch > 'Z') {
               qWarning("Karakter tidak valid: %c", ch);
               return 1;
           }
           auto it = mp.find(ch);
           if (it == mp.end()) {
               mp.emplace(ch, nextId);
               regionId[r][c] = nextId++;
           } else {
               regionId[r][c] = it->second;
           }
       }
   }
   if (nextId != n) {
       qWarning("Jumlah region harus tepat %d, ditemukan %d.", n,
nextId);
       return 1;
   }
   const int cell     = 60;
   const int gridLine = 2;
   const int size     = n * cell + (n + 1) * gridLine;
   QImage img(size, size, QImage::Format_ARGB32);
   img.fill(Qt::black);
   QPainter p(&img);
   for (int r = 0; r < n; r++) {
       for (int c = 0; c < n; c++) {
           const int x = gridLine + c * (cell + gridLine);
```

```
            const int y = gridLine + r * (cell + gridLine);
            p.fillRect(x, y, cell, cell, regionColor(regionId[r][c]));
        }
    }
    p.end();
    if (!img.save(outputPath)) {
        qWarning("Gagal menyimpan gambar: %s", qPrintable(outputPath));
        return 1;
    }
    qInfo("Berhasil disimpan: %s (%dx%d px)", qPrintable(outputPath),
size, size);
    return 0;
}
```

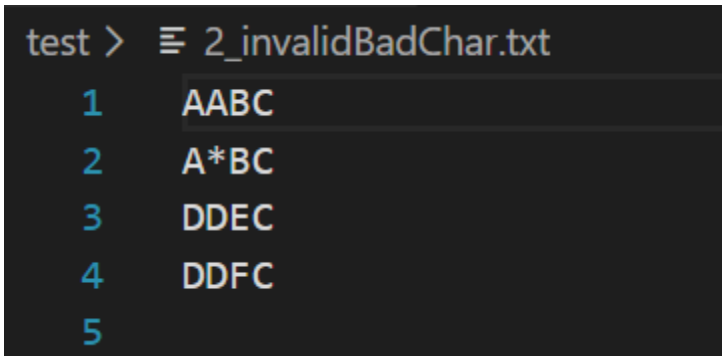
EKSPERIMEN

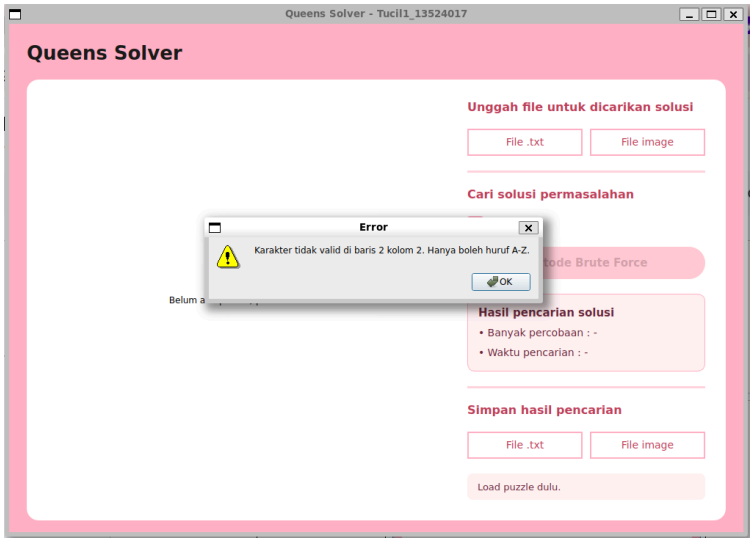
1. Kasus Input Invalid

1.1. Input kosong

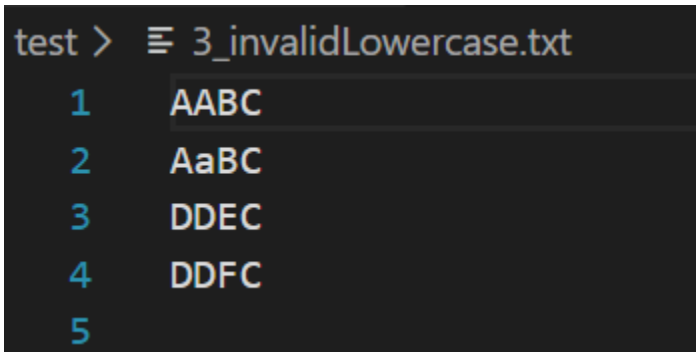
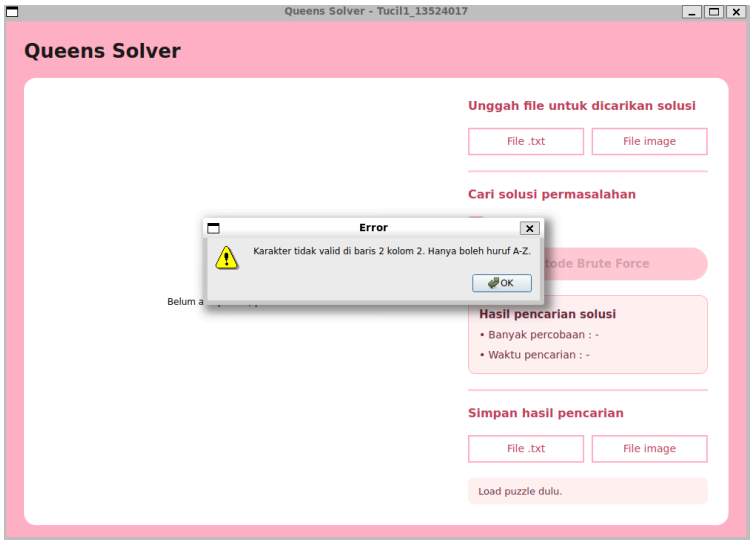
Input	
Ouput	

1.2. Input kaarakter bukan A-Z

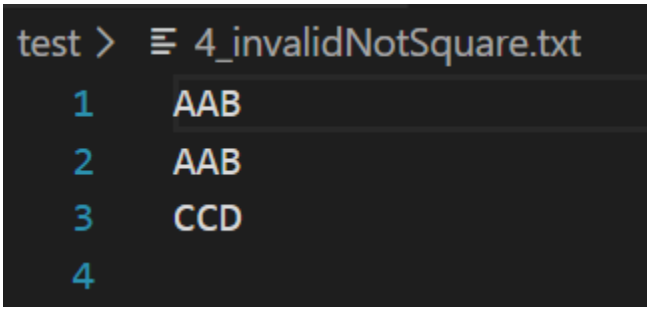
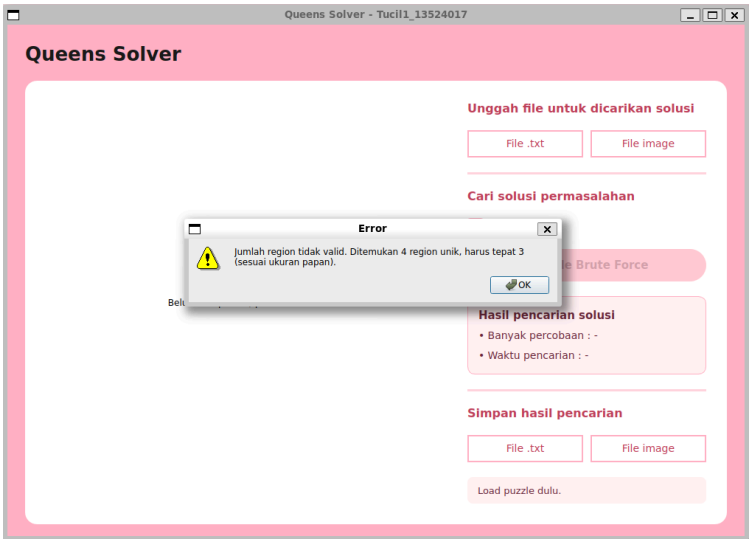
Input	

Ouput	

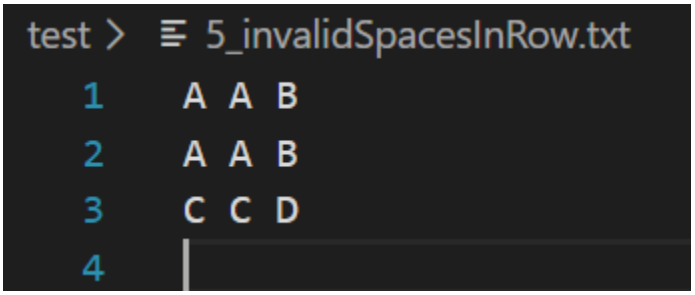
1.3. Input bukan sepenuhnya kapital

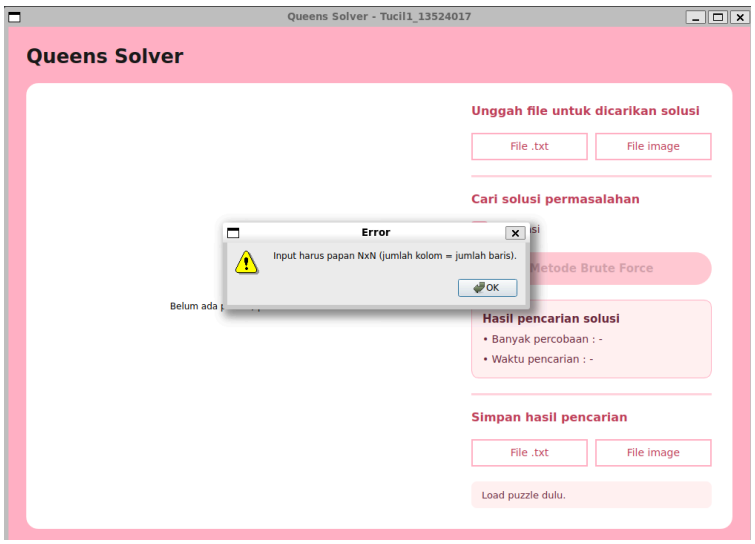
Input	
Ouput	

1.4. Input bukan berbentuk persegi

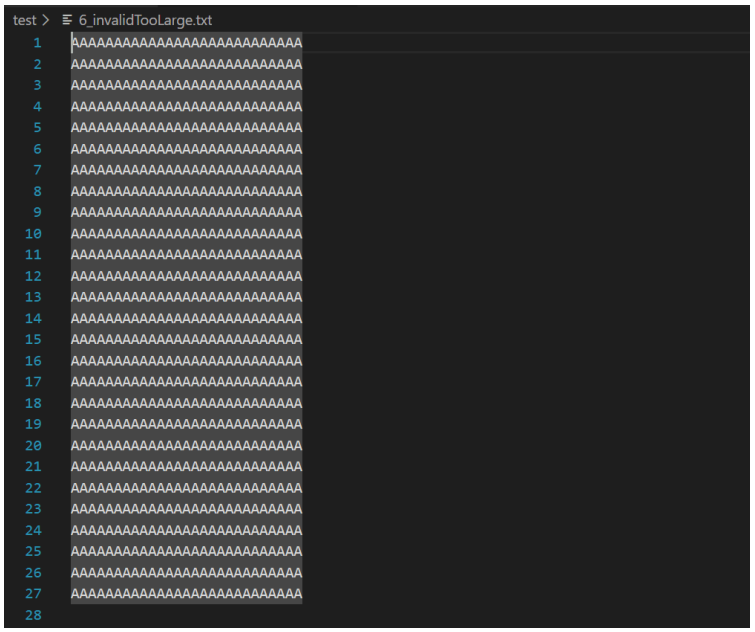
Input	
Ouput	

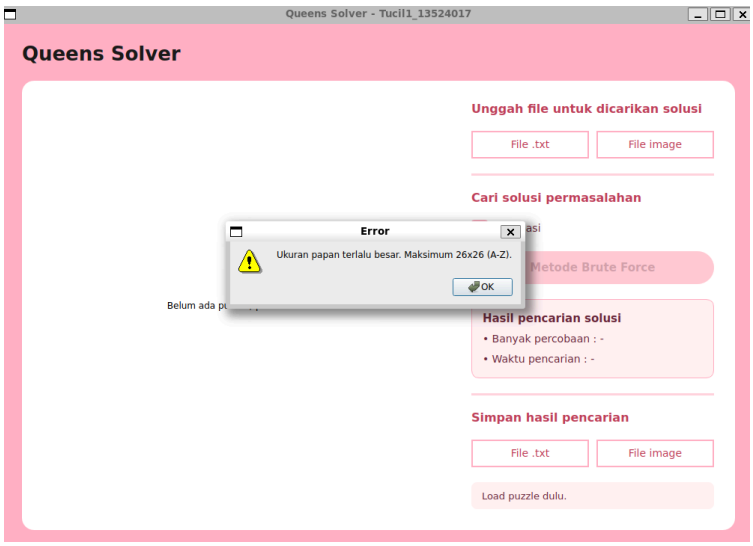
1.5. Input huruf dipisahkan oleh spasi

Input	

Ouput	

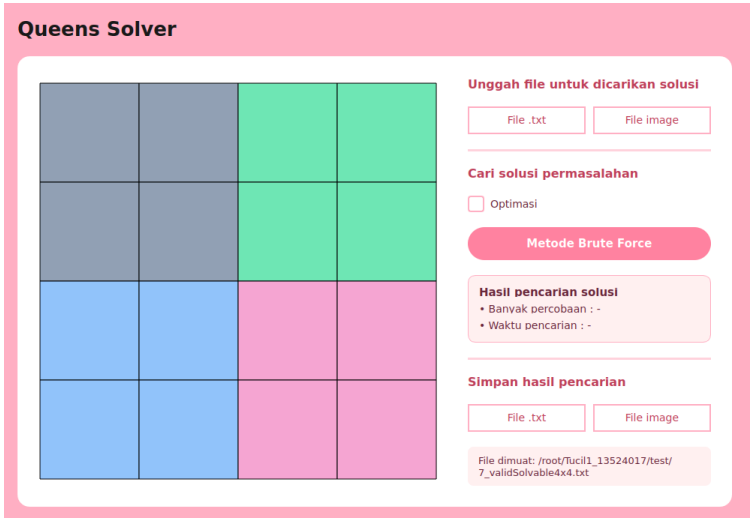
1.6. Input terlalu besar

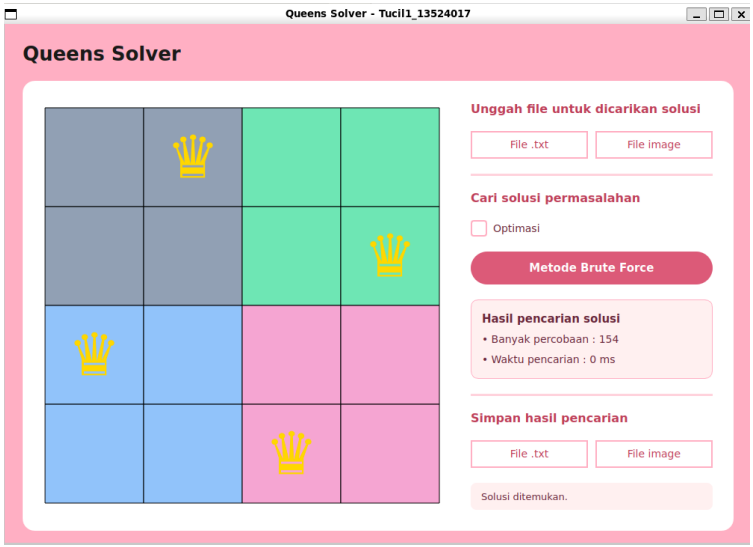
Input	

Ouput	

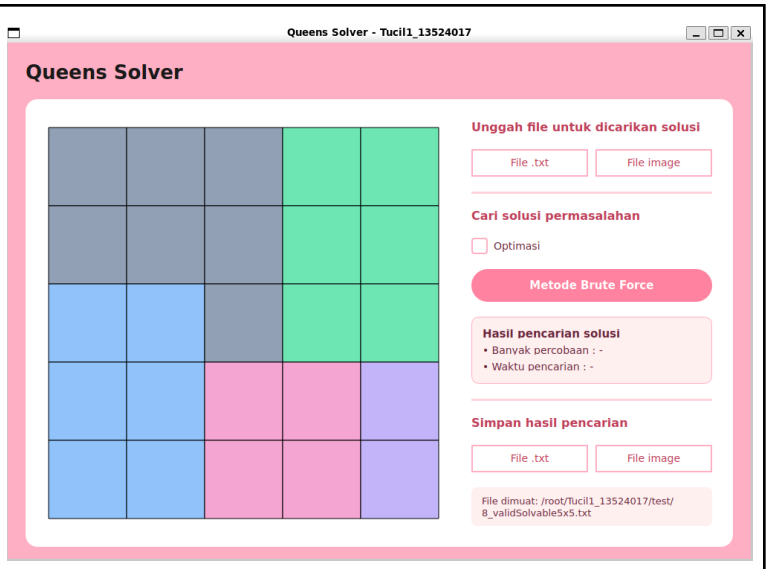
2. Kasus Input Valid dan Ada Solusi

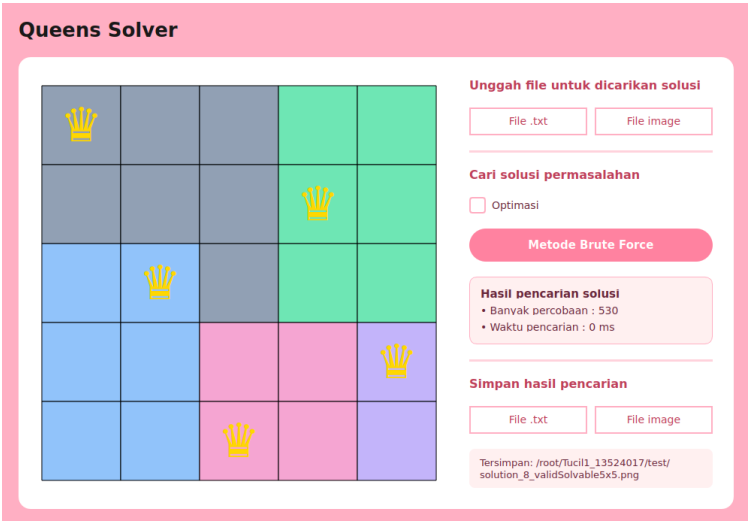
2.1. Input berukuran 4x4

Input	

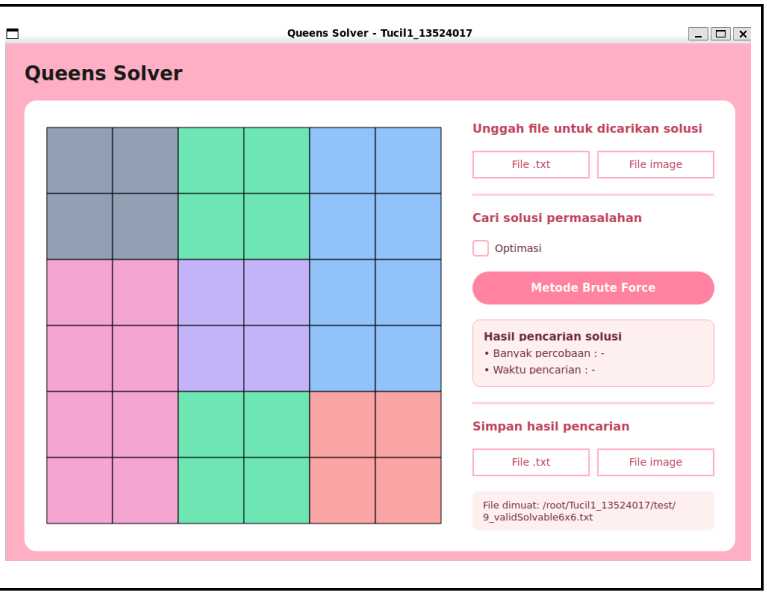
Ouput	

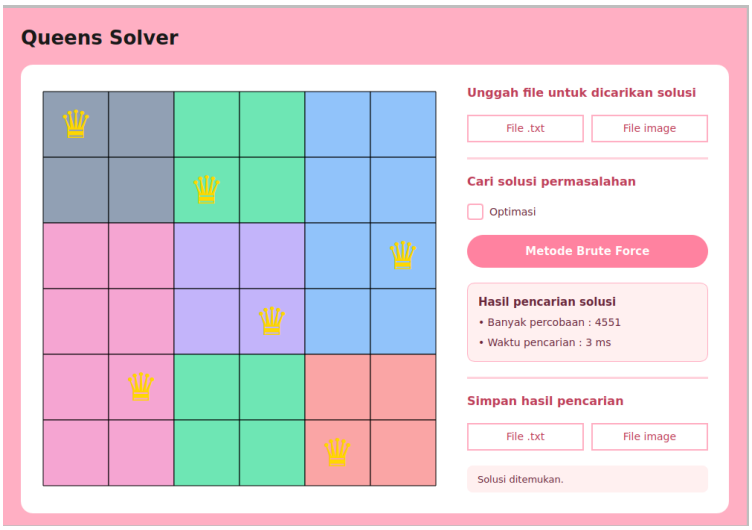
2.2. Input berukuran 5x5

Input	

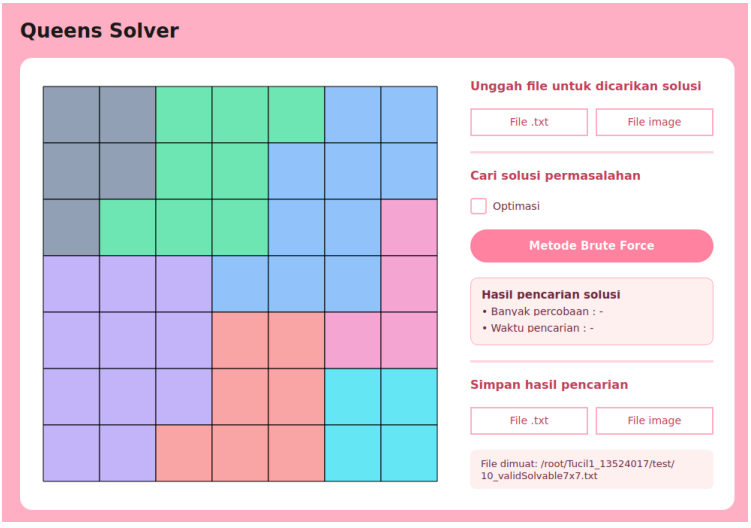
Ouput	

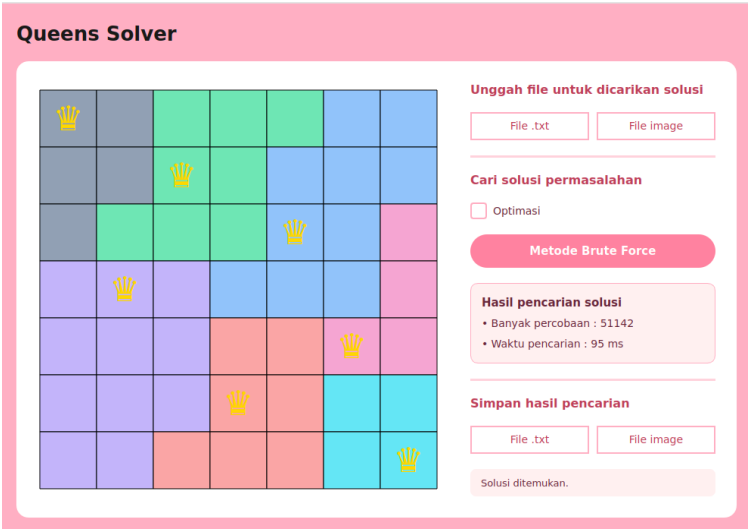
2.3. Input berukuran 6x6

Input	

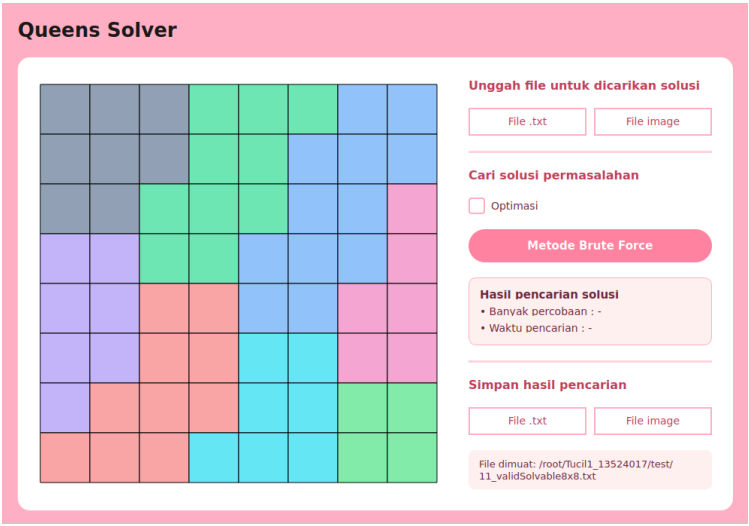
Ouput	

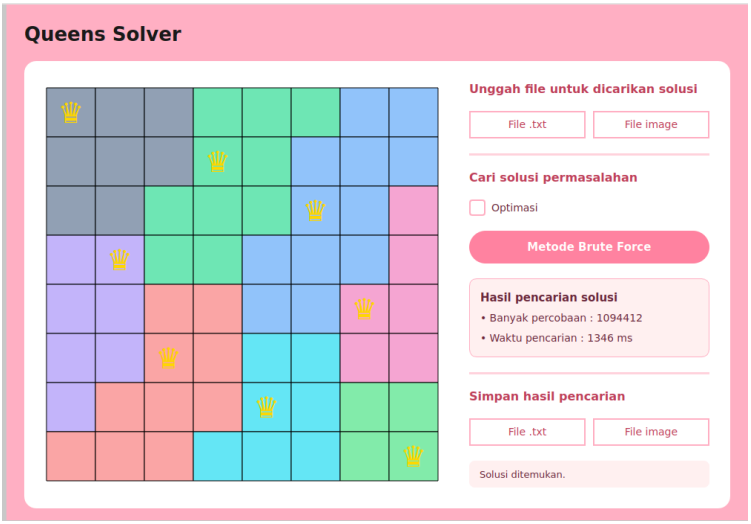
2.4. Input berukuran 7x7

Input	

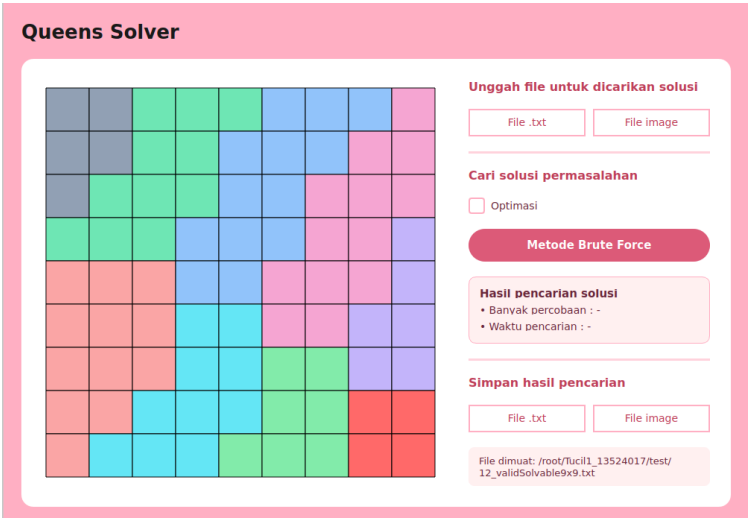
Ouput	

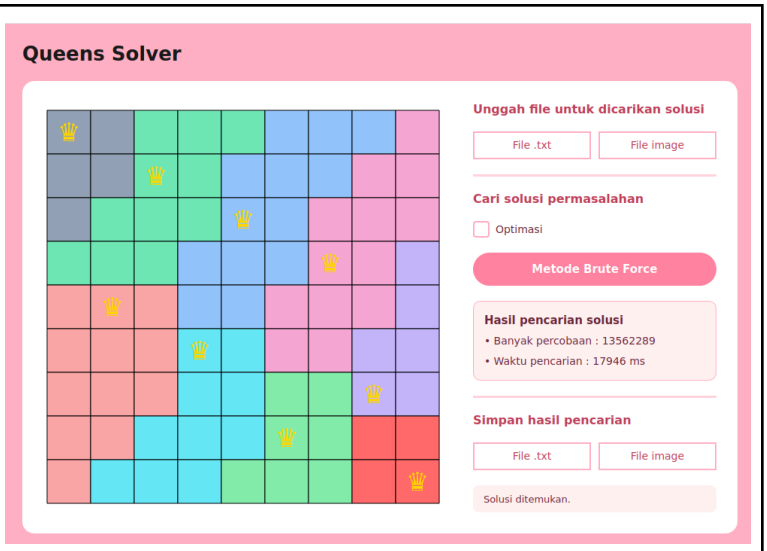
2.5. Input berukuran 8x8

Input	

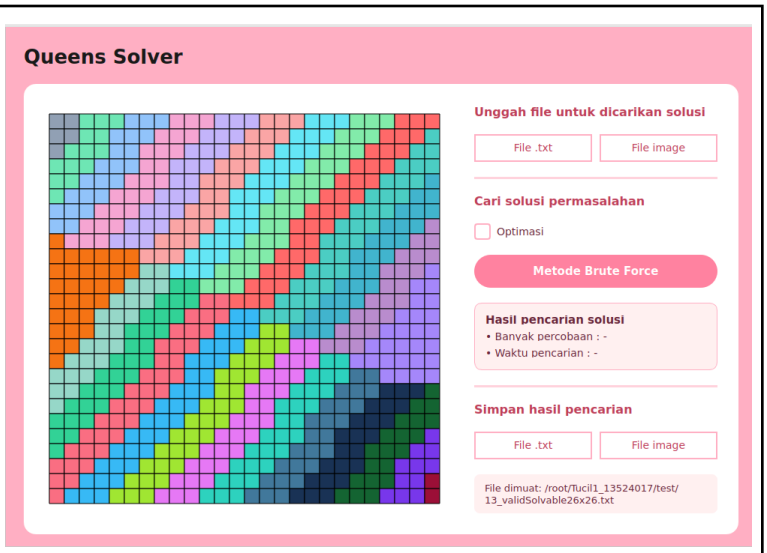
Ouput	

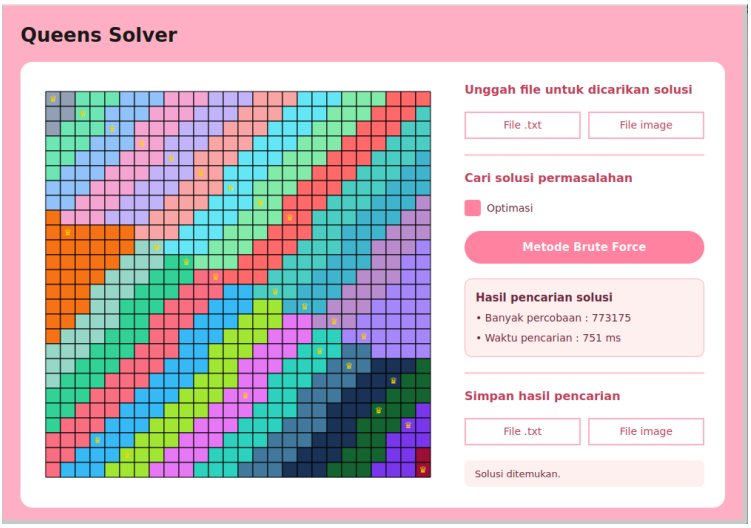
2.6. Input berukuran 9x9

Input	

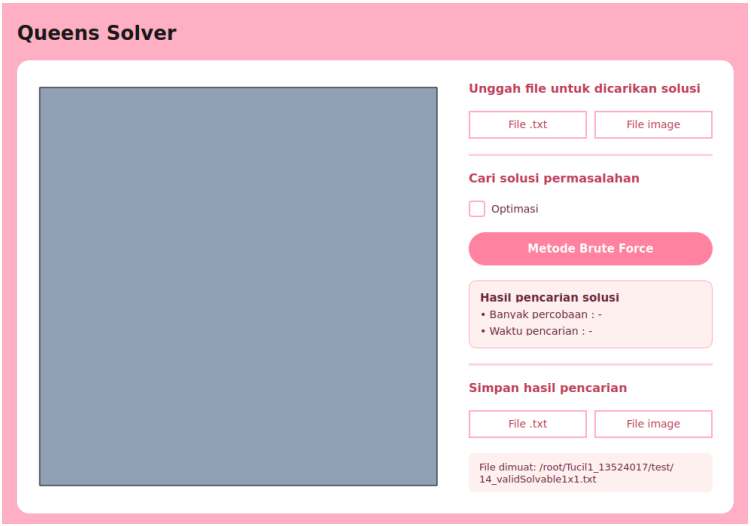
Ouput	

2.7. Input berukuran 26x26

Input	

Ouput (menggunakan optimasi)	

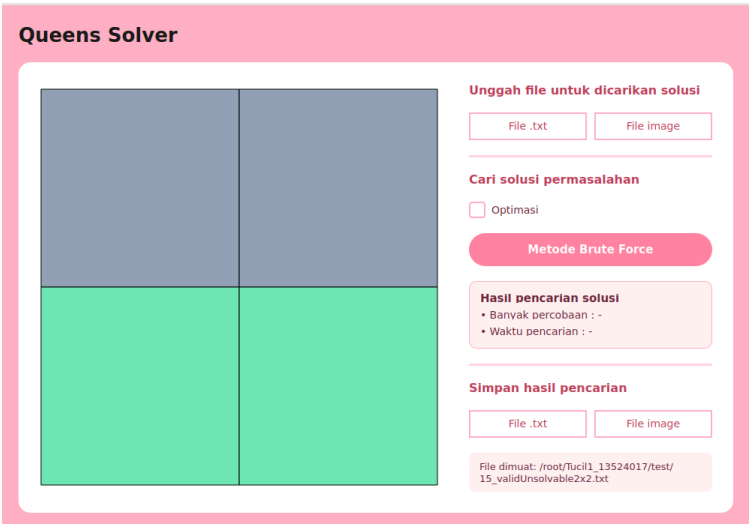
2.8. Input berukuran 1x1

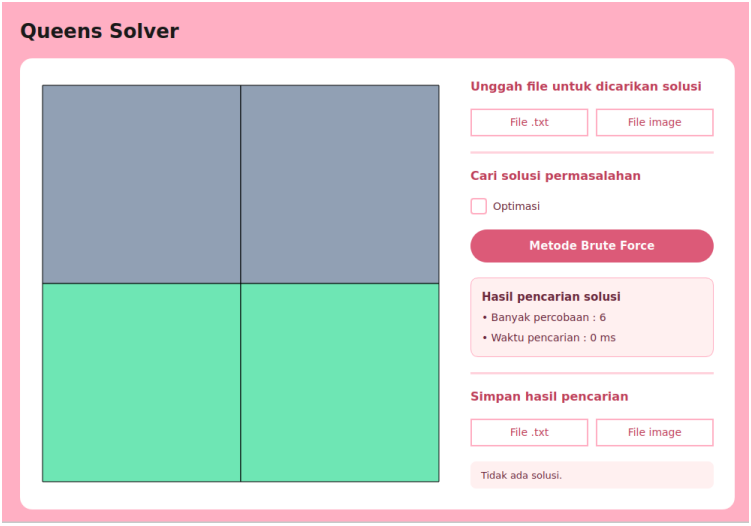
Input	

Ouput	

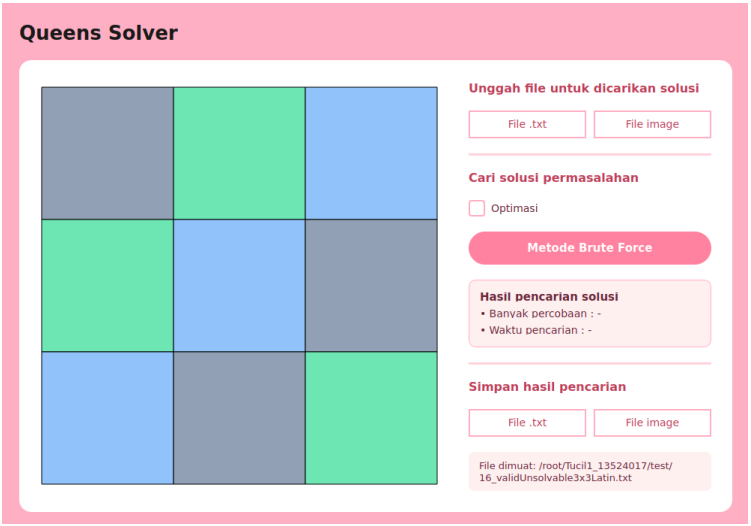
3. Kasus Input Valid dan Tidak Ada Solusi

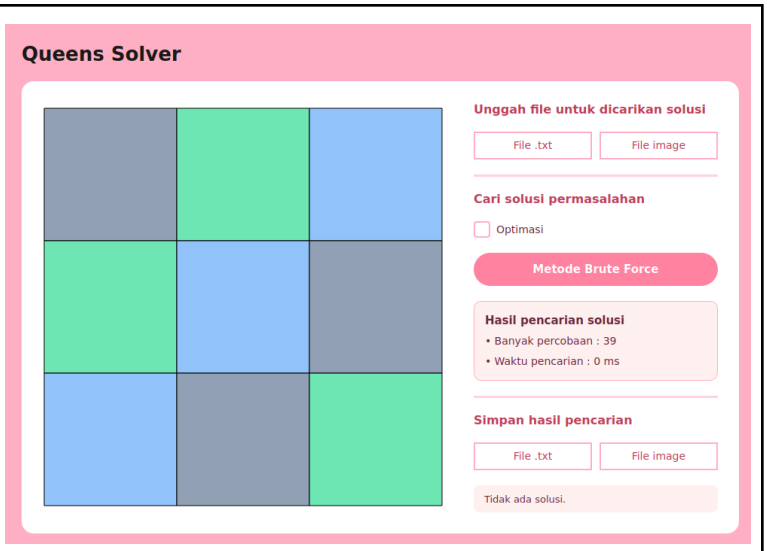
3.1. Input berukuran 2x2

Input	

Ouput	

3.2. Input berukuran 3x3

Input	

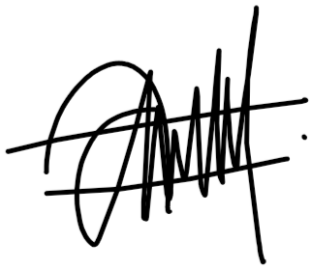
Ouput	 Queens Solver Unggah file untuk dicarikan solusi File .txt File image Cari solusi permasalahan Optimasi Metode Brute Force Hasil pencarian solusi • Banyak percobaan : 39 • Waktu pencarian : 0 ms Simpan hasil pencarian File .txt File image Tidak ada solusi.

LAMPIRAN

Repository GitHub:
zizadhrmaa/Tucil1_13524017: Tugas Kecil 1 IF2211 Strategi Algoritma : Penyelesaian Permainan Queens Linkedin

PERNYATAAN

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (Generative AI), melainkan hasil pemikiran dan analisis mandiri.



Aziza Dharma Putri